

Probabilistic Counting

Partially based on slides from:

<http://infolab.stanford.edu/~ullman/mining/2009/index.html>

Anand Rajaraman & Jeff Ullman

Counting Distinct Elements

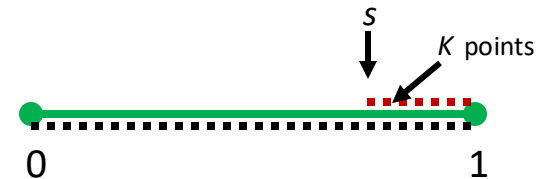
- Problem: a data stream consists of elements chosen from a set of size n (**very big!**)
How to maintain the count of the number of distinct elements seen so far?
- Obvious approach: maintain the set of elements seen (costs $O(n)$ memory!)
 - hash table, binary tree, ...
- *Use less memory (and accept loss of accuracy)*

Applications

- How many different URLs have we seen so far?
- How many different words are found among the Web pages being crawled at a site?
 - Unusually low or high numbers could indicate “artificial pages”
- How many different Web pages does each customer requests in a week?
- How many distinct elements in a column of a table?
(optimization of the join operation of two tables)
- How many distinct <source, destination> pairs through a router?
(detection of DoS attacks)

Simple Idea: MinTopK Estimate

- Hash incoming objects into doubles from the interval $[0, 1]$ and count them, shrinking the interval if needed
- Due to limited memory, maintain only the K biggest values (“TopK”), say, $K = 1000$
- Let s denote the minimum of our set (MinTopK)



- The number of distinct elements $c \approx K/(1 - s)$
 - e.g. $K = 1000, s = 0.9$: $c = \frac{1000}{1-0.9} = \frac{1000}{0.1} = 10,000$
 - $K = 1000, s = 0$: $c = \frac{1000}{1-0} = \frac{1000}{1} = 1000$
 - $K = 1000, s = 0.999$: $c = \frac{1000}{1-0.999} = \frac{1000}{0.001} = 1,000,000$

Flajolet-Martin Approach

- Key idea:
 - hash passing elements into short bitstrings
 - store only the length of the longest tail of 0's
 - the more distinct elements, the longer the longest tail of 0's
- Pick a hash function h that maps each of the m elements to $\lg m$ bits
- For each stream element a , let $r(a)$ be the number of trailing 0's in $h(a)$
- Record $R =$ the maximum $r(a)$ seen
- Estimate the number of distinct elements as $2^R / \phi$, where $\phi = 0.77351$

hash function
randomizes

$X_1 \rightarrow 1000111\mathbf{0} \rightarrow 1$	1
$X_2 \rightarrow 0101001\mathbf{0} \rightarrow 1$	1
$X_3 \rightarrow 10011011 \rightarrow 0$	1
$X_4 \rightarrow 00101\mathbf{000} \rightarrow 3$	3
$X_5 \rightarrow 0101111\mathbf{0} \rightarrow 1$	3

#bits #bits

$N \rightarrow \lg N \rightarrow \lg \lg N$

e.g. $\lg \lg 1,000,000 < 5$

$$2^R / \phi = 2^3 / 0.77351 = 10.3$$

Intuition

- The more different elements we see, the more different hash-values
- As we see more different hash-values, it becomes more likely that one of these values will be “unusual”
- Ending in many 0’s is unusual, so that is a good measure
 - other options exist

Why does it Work?

- How likely is it that for a random number, you come across at least r 0's?
- The probability that a given $h(a)$ ends in at least r 0's is

$$2^{-r}$$

- If there are m different elements, the probability that none have r 0's (so $R \geq r$) is

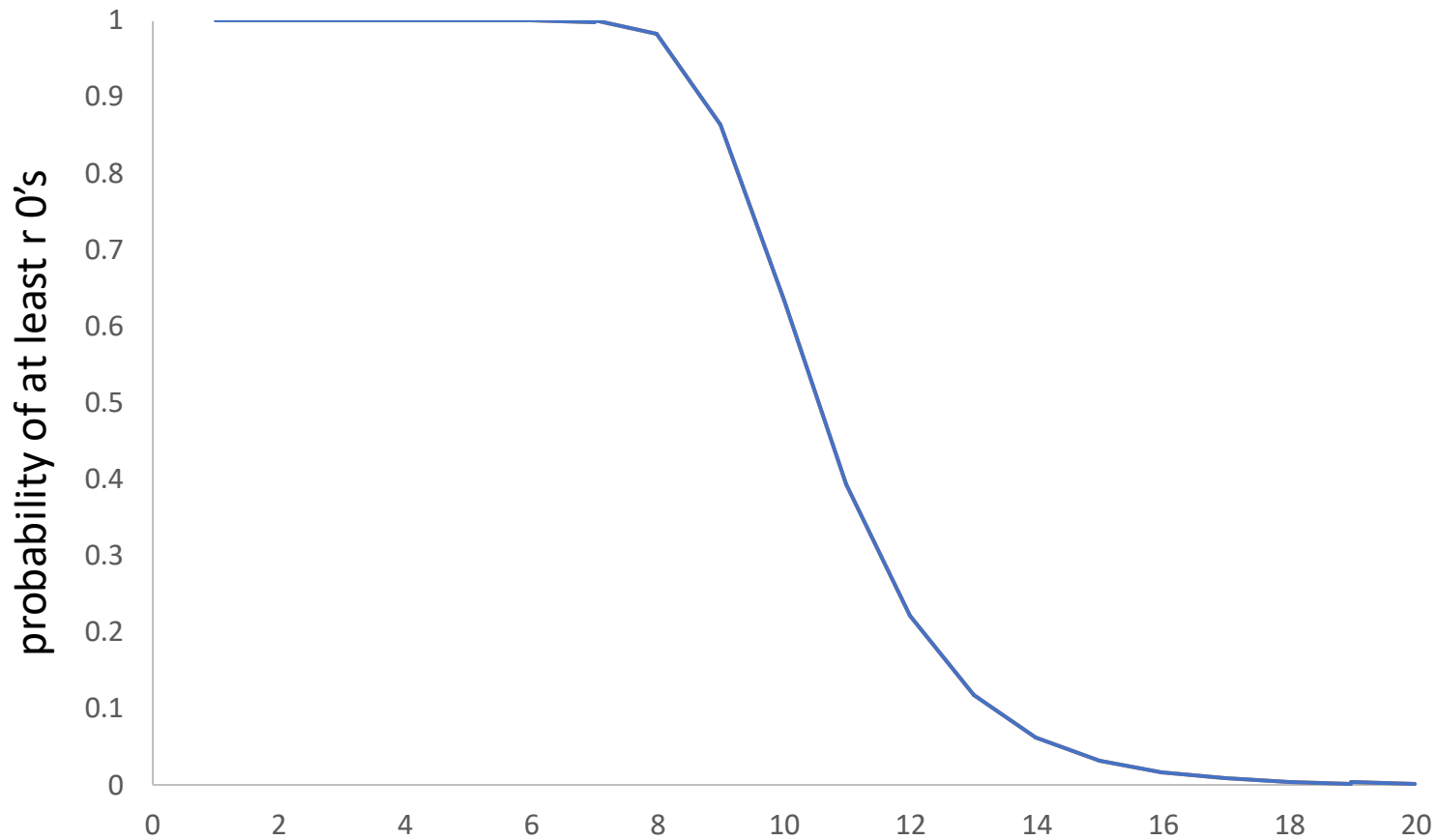
$$1 - (1 - 2^{-r})^m$$

Probability all $h(a)$'s
end in fewer than r 0's

Probability an $h(a)$
ends in fewer than
 r 0's

Why does it Work?

- Since 2^{-r} is small, $1 - (1 - 2^{-r})^m \approx 1 - e^{-m/2^r}$
- If $2^r \gg m$, then $1 - (1 - 2^{-r})^m \approx 1 - 1 = 0$ (unlikely to be much larger)
- If $2^r \ll m$, then $1 - (1 - 2^{-r})^m \approx 1 - 0 = 1$ (unlikely to be much smaller)
- Thus, 2^R will almost always be around m

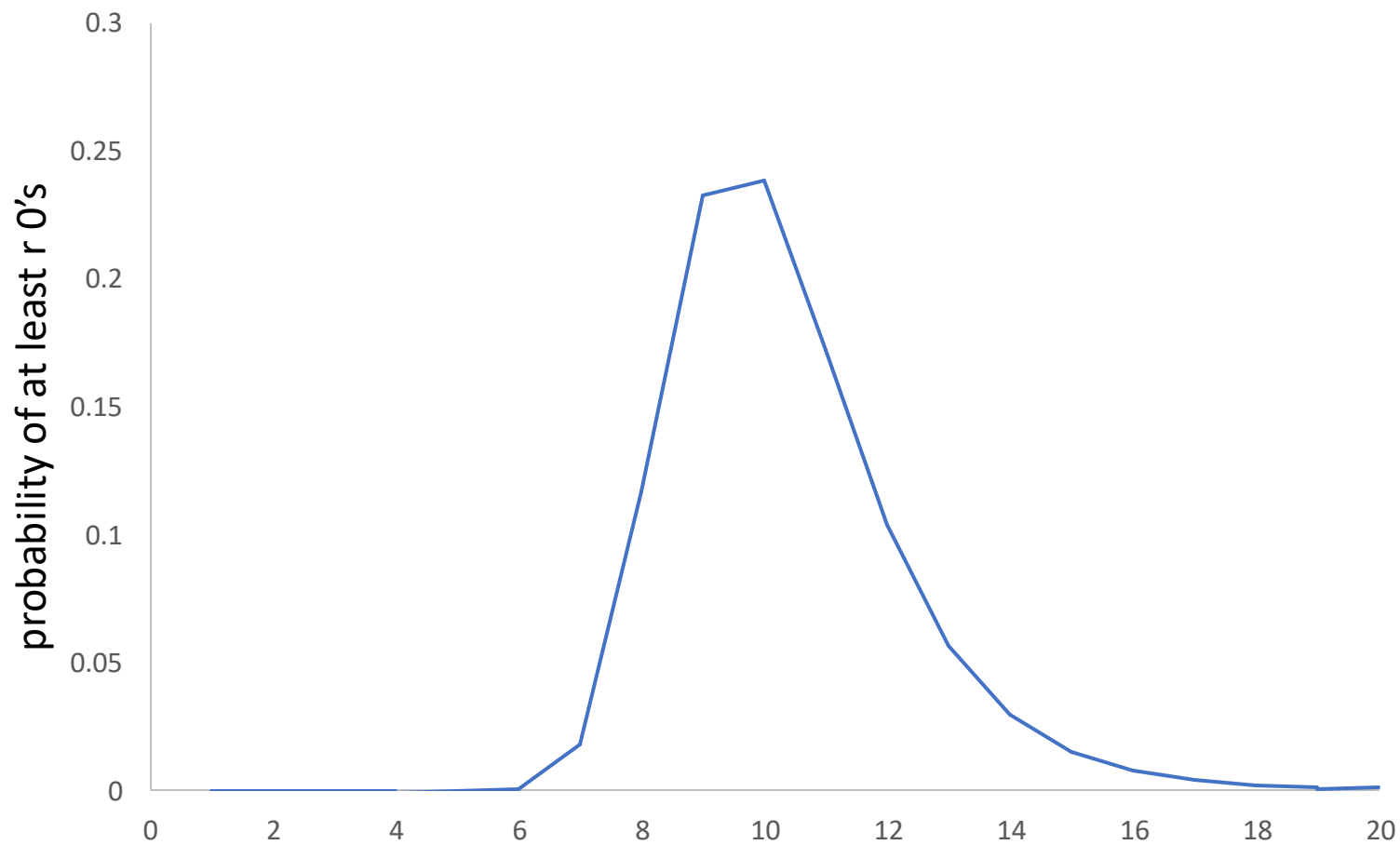


$m = 1024$

$r = \lg 1024 = 10$

distinct elements

expected longest tail of 0's



$m = 1024$
 $r = \lg 1024 = 10$

distinct elements
expected longest tail of 0's

When It Doesn't Work

- 2^R is always a power of 2...
- Bad luck can result in huge errors...
- Workaround: run in parallel several copies of this algorithm, using different hash functions and average the results
- How do we average results?
 - Mean: what if one very large value?
 - e.g. $\text{mean}(2^9, 2^9, 2^{10}) = 682.66$
 - Median: always a power of 2, so considerable error!
 - e.g. $\text{median}(2^9, 2^9, 2^{10}) = 2^9 = 512$

Durand, Flajolet: the LogLog algorithm

- Key ideas: *stochastic averaging + calibration*
 - Partition your samples into $n = 2^l$ groups, using the first l bits of the hash function as a selector
 - Calculate $R_1, \dots, R_n$, (R_i for group i) and return:

$$a_n \cdot n \cdot 2^{\text{mean}(R_1, \dots, R_n)}$$

where a_n is correction factor (precomputed)

- $n = 1024 \rightarrow$ relative error 3% -4%

Complexity

- Required memory
 - We work with bit strings of length $\lg n$
 - We only have to maintain the length of the longest tail of 0's - $\lg(\lg n)$
 - almost nothing:
 - 5 bits $\rightarrow 2^{32} = 10^9$ objects
 - 8 bits $\rightarrow 2^{256} = 10^{77}$ objects
 - Multiply it by the number of hash functions:
 - $1024 \cdot 5 = 640$ bytes of memory
- Time
 - processing an element requires computing values of **1** hash function (linear in the length of input)

Example: Entire Shakespeare's Work

- 4 bits each (1/2 byte, 1 *nibble*)
- 256 registers
- 128 bytes
- $2^{2^4} = 2^{16} = 65,536 = 10^5$ maximum distinct elements

```
ghfffghfghgghgggghghheehfhfhghgghghghfhgffffhhhiigfhhffgfiihfhhh  
igigighfghihfffghigihghigfhhgeegeghgghhhgghhhfidiigihighihehhfgg  
hfgighigffghdieghhhggghhfhghhfiieffghghihifgggffihgihfggighgiiif  
fjgfgjhhjiifhjgehgghfhfhjhiggghghihigghhihihgiighghlglgjgjjmfl
```

The LOGLOG Algorithm with $m = 256$ condenses the whole of Shakespeare's works to a table of 256 "small bytes" of 4 bits each. The estimate of the number of distinct words is here $n^\circ = 30897$ (true answer: $n = 28239$), i.e., a relative error of +9.4%.

Recommended papers:

- P. Flajolet:
Counting by Coin Tossings
- M. Durand and P. Flajolet:
Loglog Counting of Large Cardinalities
- A. Metwally, D. Agrawal and A. El Abbadi:
Why Go Logarithmic if We Can Go Linear?